

API – Coding Challenge – 2

Sudharsana I

26-06-2025

Summary of the project

Task Management API

- **Core Functionality:**
 - **Database:** MySQL for task storage.
 - **Task Entity:** Defines task structure.
 - **Task Repository:** Handles task CRUD.
 - **DTOs:** For API request/response.
 - **Task Service:** Business logic.
 - **Task Controller:** REST endpoints for tasks.
 - **Global Error Handling:** Consistent error responses.
- **JWT Authentication:**
 - **User Storage:** Database-backed User entity and Repository.
 - **Password Security:** BCrypt hashing.
 - **Custom UserDetailsService:** Loads user data.
 - **JWT Utility:** Token generation and validation.
 - **JWT Filter:** Validates tokens on requests.
 - **Auth Controller:** /register and /login endpoints.
 - **Security Config:** Protects /api/tasks/** with JWT, exposes auth endpoints, disables CSRF, enables CORS.
 - **Security Error Handling:** Provides 401/403 responses.

Creation of Spring Folder Structure

The screenshot shows the Spring Initializr web application at <https://start.spring.io>. The interface is divided into several sections:

- Project:** Includes radio buttons for **Gradle - Groovy**, **Gradle - Kotlin**, and **Maven** (selected).
- Language:** Includes radio buttons for **Java** (selected), **Kotlin**, and **Groovy**.
- Spring Boot:** Includes radio buttons for versions **4.0.0 (SNAPSHOT)**, **3.5.4 (SNAPSHOT)**, **3.5.3** (selected), **3.4.8 (SNAPSHOT)**, **3.4.7**, and **3.3.13**.
- Project Metadata:** Includes fields for **Group** (com.hexaware.tasks), **Artifact** (task-manager-api), **Name** (task-manager-api), and **Description** (API for managing tasks).
- Dependencies:** A list of dependencies with checkboxes: **Spring Web** (WEB), **Spring Data JPA** (SQL), **Lombok** (DEVELOPER TOOLS), and **Spring Security** (SECURITY). An **ADD DEPENDENCIES... CTRL + B** button is present.
- Buttons:** At the bottom, there are buttons for **GENERATE CTRL + G**, **EXPLORE CTRL + SPACE**, and an ellipsis button.

Creation of Database and Needed tables

The screenshot shows the MySQL Workbench interface with the following components:

- Navigator:** A tree view on the left showing the database structure, including schemas like **apdb**, **car**, **carconnectdb**, **currentmanagement**, **clustrex**, **college**, **cozy_haven_stay_app**, **crimemanagement**, **headb1**, **hospitalmanagement**, **java_stock**, **sys**, **taskdb**, **taxcalculation**, and **techshop**.
- SQL Editor:** The central area contains two SQL queries:

```
1 CREATE DATABASE taskdb;
2
3 USE taskdb;
4
5 CREATE TABLE tasks (
6     id BIGINT AUTO_INCREMENT PRIMARY KEY,
7     title VARCHAR(255) NOT NULL,
8     description TEXT,
9     due_date DATE,
10    priority VARCHAR(50) NOT NULL,
11    status VARCHAR(50) NOT NULL
12 );
13
14 CREATE TABLE users (
15     id BIGINT AUTO_INCREMENT PRIMARY KEY,
16     username VARCHAR(255) NOT NULL UNIQUE,
17     password VARCHAR(255) NOT NULL,
18     -- ... (truncated) ...
19 );
```
- SQLAdditions:** A sidebar on the right showing the **VARCHAR Syntax:**

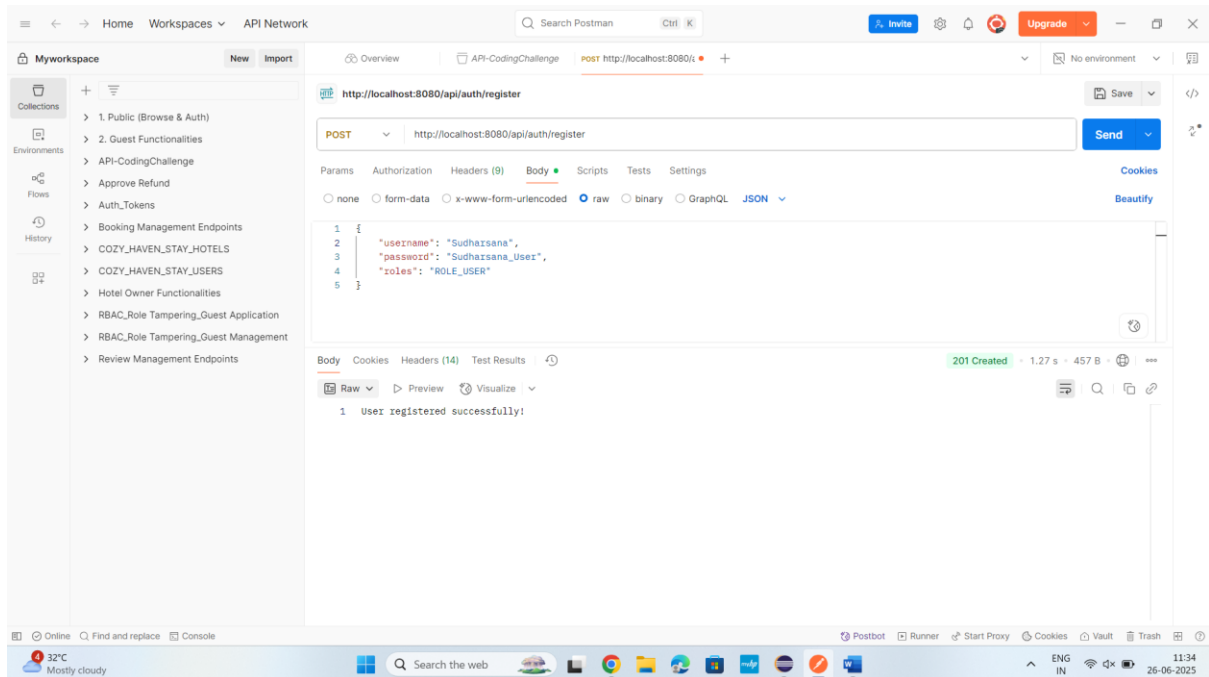
VARCHAR Syntax:

[NATIONAL] VARCHAR(M)
[CHARACTER SET charset_name] [COLLATE collation_name]

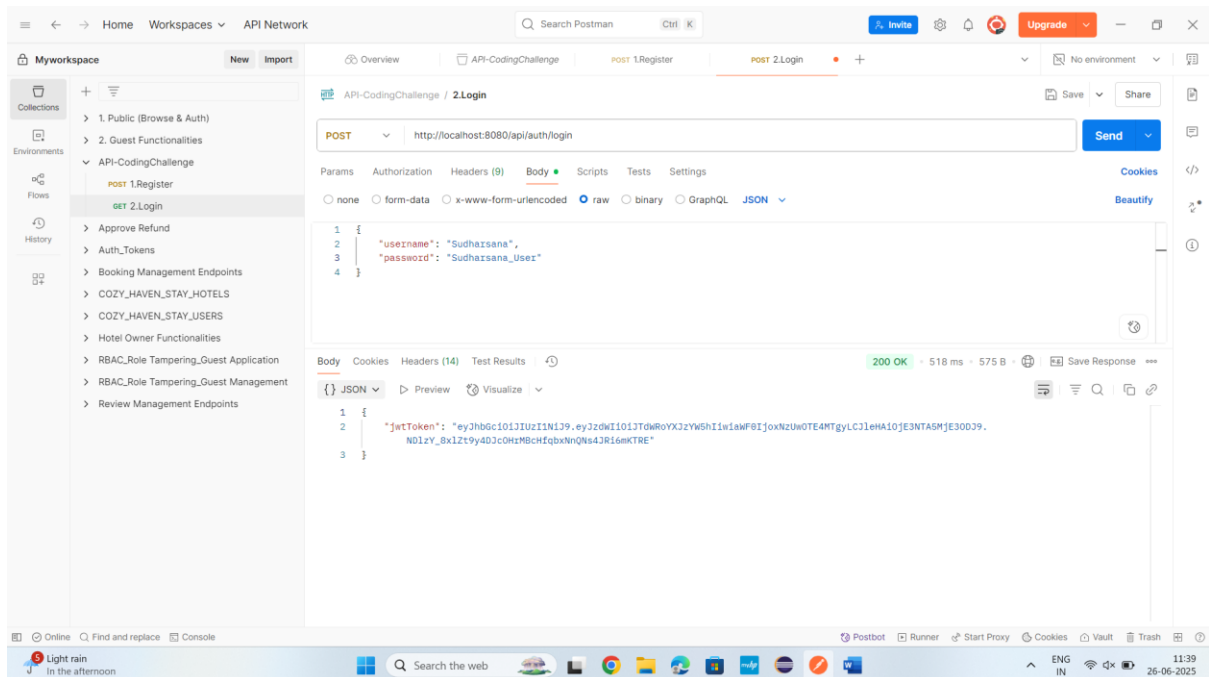
A variable-length string. **M** represents the maximum column length in characters. The range of **M** is 0 to 65,535. The effective maximum length of a **VARCHAR** is subject to the maximum row size (65,535 bytes, which is shared among all columns) and the character set used. For example, **UTF8** characters can require up to three bytes per character, so a **VARCHAR** column that uses the **UTF8** character set can be declared to be a maximum of 21,844.
- Output:** A section at the bottom showing the results of the SQL execution, including a table with columns **Time** and **Action**.

Postman Output:

1.Register



2.Login



3.Add a new task.

The screenshot shows the Postman interface with a workspace named 'Myworkspace'. The API is 'API-CodingChallenge' and the endpoint is 'POST http://localhost:8080/api/tasks'. The request body is a JSON object:

```
{  "title": "Complete all fixes",  "description": "Ensure package structure and config are perfect.",  "dueDate": "2025-06-30",  "priority": "HIGH",  "status": "IN_PROGRESS"}
```

. The response is a 201 Created status with a 449 ms response time and 594 B body. The response body is a JSON object:

```
{  "id": 1,  "title": "Complete all fixes",  "description": "Ensure package structure and config are perfect.",  "dueDate": "2025-06-30",  "priority": "HIGH",  "status": "IN_PROGRESS"}
```

4.Retrieve all tasks.

The screenshot shows the Postman interface with a workspace named 'Myworkspace'. The API is 'API-CodingChallenge' and the endpoint is 'GET http://localhost:8080/api/tasks'. The request is a GET request with a Bearer Token authorization header. The response is a 200 OK status with a 33 ms response time and 591 B body. The response body is a JSON array:

```
[  {    "id": 1,    "title": "Complete all fixes",    "description": "Ensure package structure and config are perfect.",    "dueDate": "2025-06-30",    "priority": "HIGH",    "status": "IN_PROGRESS"  }]
```

5.Retrieve a single task by its ID.

The screenshot shows the Postman interface with a workspace named 'Myworkspace'. The left sidebar displays a collection of API endpoints under 'API-CodingChallenge'. The selected endpoint is 'GET 5.Retrieve a Single Task by ID'. The main panel shows the request details: Method 'GET', URL 'http://localhost:8080/api/tasks/1', and a status of '200 OK'. The response body is a JSON object representing a task.

```
GET http://localhost:8080/api/tasks/1
```

Params Authorization Headers (7) Body Scripts Tests Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers (14) Test Results

200 OK · 33 ms · 589 B

```
{
  "id": 1,
  "title": "Complete all fixes",
  "description": "Ensure package structure and config are perfect.",
  "dueDate": "2025-06-30",
  "priority": "HIGH",
  "status": "IN_PROGRESS"
}
```

6.Update an existing task.

The screenshot shows the Postman interface with a workspace named 'Myworkspace'. The left sidebar displays a collection of API endpoints under 'API-CodingChallenge'. The selected endpoint is 'PUT 6.Update an Existing Task'. The main panel shows the request details: Method 'PUT', URL 'http://localhost:8080/api/tasks/1', and a status of '200 OK'. The request body is a JSON object representing the updated task.

```
PUT http://localhost:8080/api/tasks/1
```

Params Authorization Headers (10) Body Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

```
{
  "id": 1,
  "title": "All systems go!",
  "description": "Updated The task manager API is now fully operational with JWT authentication!",
  "dueDate": "2025-06-26",
  "priority": "HIGH",
  "status": "COMPLETED"
}
```

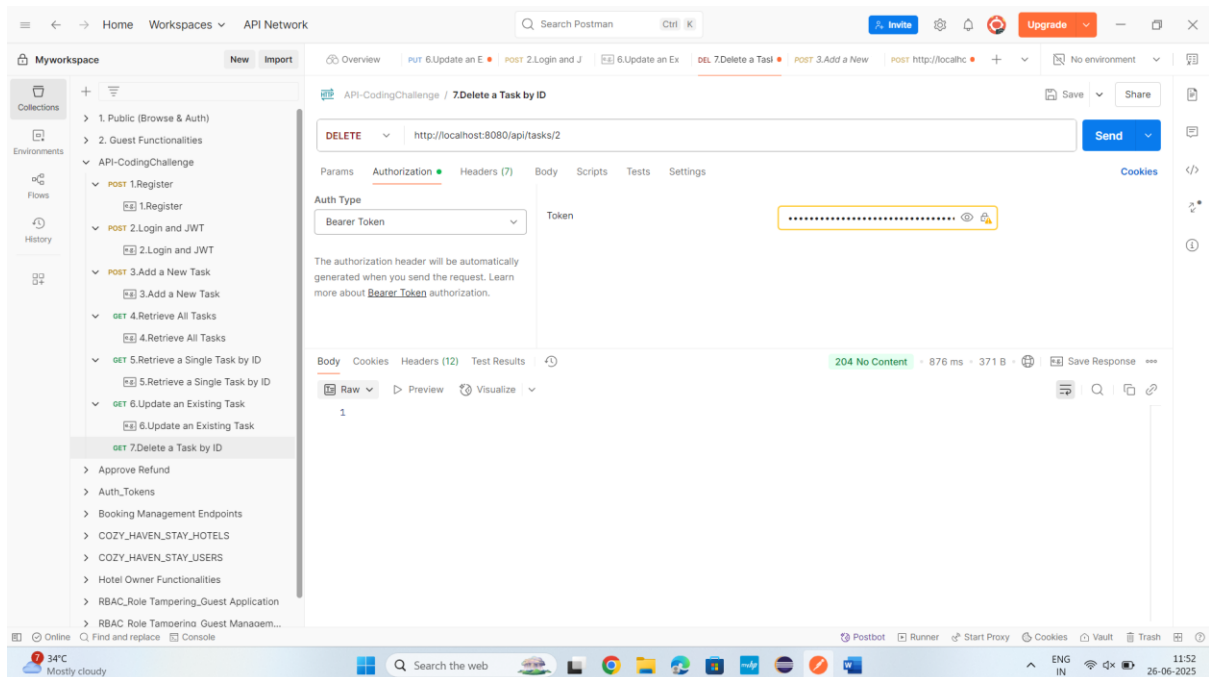
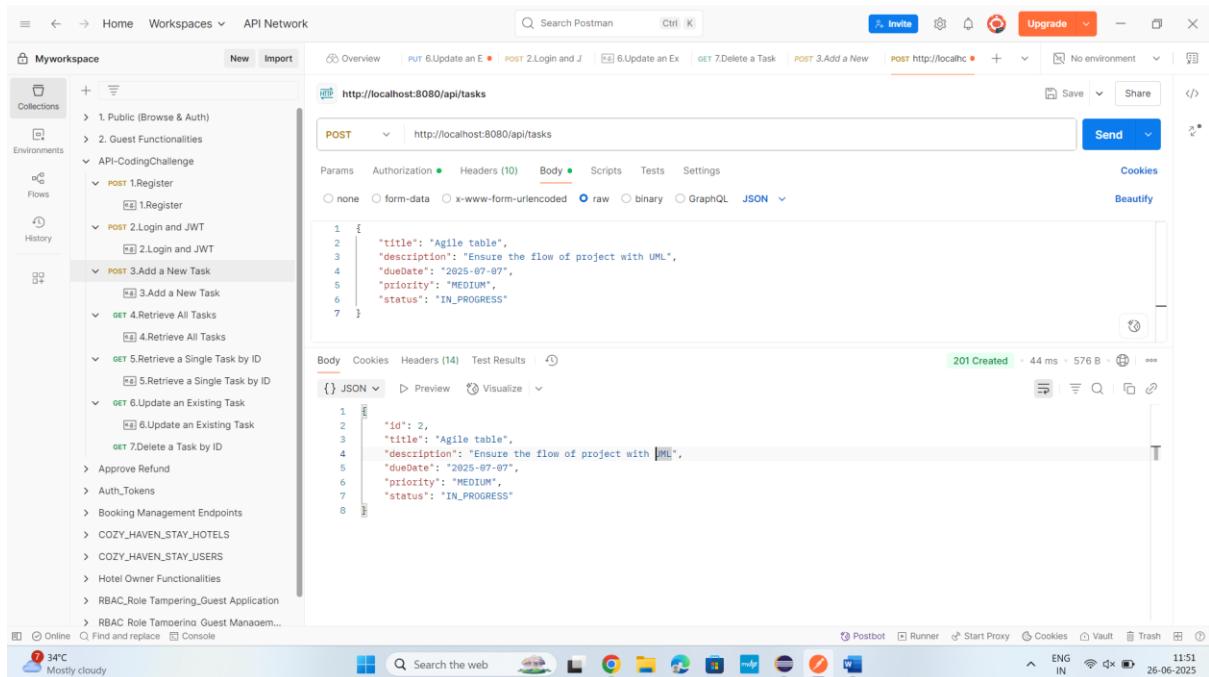
Body Cookies Headers (14) Test Results

200 OK · 61 ms · 614 B

```
{
  "id": 1,
  "title": "All systems go!",
  "description": "Updated The task manager API is now fully operational with JWT authentication!",
  "dueDate": "2025-06-26",
  "priority": "HIGH",
  "status": "COMPLETED"
}
```

7.Delete a task by its ID.

Added one more task with id 2 and deleted the same



8. Without Authorization header

The screenshot shows the Postman interface with a workspace named "Myworkspace". The left sidebar displays a collection of API requests under "API-CodingChallenge". The main panel shows a GET request to "http://localhost:8080/api/tasks". The response is a "401 Unauthorized" status, indicating that the request is missing the necessary authorization header.

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (13) Test Results

Raw Preview Visualize

1

401 Unauthorized 34 ms 392 B

Postbot Runner Start Proxy Cookies Vault Trash

34°C Mostly cloudy Search the web ENG IN 11:55 26-06-2025